

Análisis de Series de Tiempo II

Temporal Convolutional Networks y Arquitecturas Modernas

MSc. Fernando Silva



Agenda

- Repaso
- CNN para Series Temporales
- WaveNet
- Temporal Convolutional Network
- Descomposición Temporal
- N-BEATS y N-HiTS

Repaso

Multi-step $\neq$ Multivariado $\neq$ Multi-output

Multi-step (Clase 1)

¿Cuántos pasos hacia el futuro queremos predecir?

Estrategias recursiva, directa, **MIMO**

Multivariado (input)

¿Cuántas variables observa el modelo?

Dimensión del input por paso, el modelo observa **d** variables en cada instante

Multi-output (target)

¿Cuántas variables queremos predecir?

Predecir varias series a la vez, $\hat{y}_{t+1} \in \mathbb{R}^k$; **VAR** es el caso clásico lineal

- Veremos multivariado en input con target escalar:

$$X \in \mathbb{R}^{L \times d} \quad a \quad \hat{y} \in \mathbb{R}$$

¿Por qué existe el enfoque global?

Enfoque local:

Los modelos clásicos, como ARIMA o ETS, entrenan un modelo independiente para cada serie temporal, ya que cada una se considera un problema diferente.

Enfoque global:

En lugar de entrenar miles de modelos, se entrena un único modelo con todas las series, compartiendo los mismos parámetros y aprendiendo patrones comunes entre ellas.

- El enfoque global surge por un problema de escala: empresas con miles o millones de series temporales no pueden mantener un modelo distinto para cada una.
- Las redes neuronales tienen millones de parámetros y necesitan grandes cantidades de datos. Una sola serie suele ser insuficiente, pero al entrenar con muchas series el modelo dispone de información suficiente para aprender.
- En series temporales, los datos suelen crecer más en número de series que en longitud de cada serie. Por eso, el enfoque global no solo reduce costos de entrenamiento, sino que hace posible aplicar deep learning de forma efectiva.

Forecasting Jerárquico

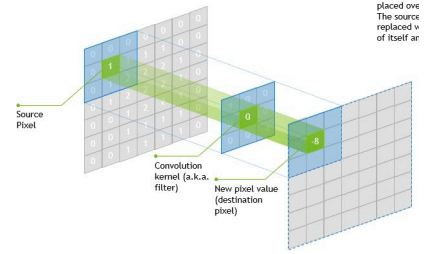
- El forecasting jerárquico es un enfoque de predicción de series de tiempo donde los datos tienen una estructura organizada en diferentes niveles de agregación, y el objetivo es generar pronósticos que sean coherentes entre todos los niveles de la jerarquía.
- No se predice una sola serie temporal aislada, sino un conjunto de series relacionadas mediante una estructura jerárquica.



CNN para Series Temporales

De imágenes a secuencias

- Una CNN clásica desliza un filtro sobre una imagen (2D): alto $\times$ ancho $\times$ canales.
- Para series temporales usamos convoluciones 1D: el filtro desliza solo sobre el eje temporal.
- Cada canal de entrada puede ser una variable distinta (multivariado): la "profundidad" del filtro cubre todas las variables en un mismo paso temporal.
- En lugar de recorrer la secuencia paso a paso (como una RNN), la CNN mira ventanas locales de la secuencia en paralelo.



Convolución 1D

- Dada una serie:

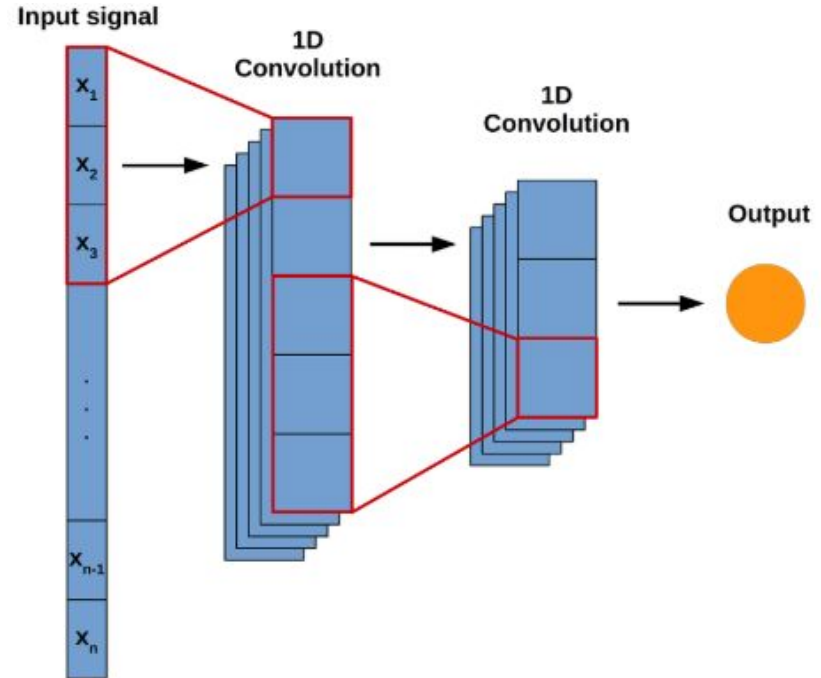
$$x = (x_1, \dots, x_T)$$

y un filtro de tamaño k , la salida en el paso t es:

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-i}$$

k = tamaño de kernel (cuántos pasos pasados “mira” el filtro)

- Los pesos w se comparten en toda la secuencia, por eso son muchos menos parámetros que una capa densa



El problema de la convolución en series temporales

- Una convolución estándar (no causal) usa contexto pasado y futuro:

$$y_t \text{ depende de } x_{t-1}, x_t, x_{t+1}$$

- Para forecasting esto es **data leakage**:
en el momento t no conocemos x_{t+1}
- Además, el **receptive field** de una convolución estándar crece muy lento con la profundidad (linealmente), lo que obliga a apilar demasiadas capas para capturar dependencias largas
- Estos **dos problemas** motivan las **dos modificaciones** que dan origen a la TCN: convoluciones causales y dilatadas.

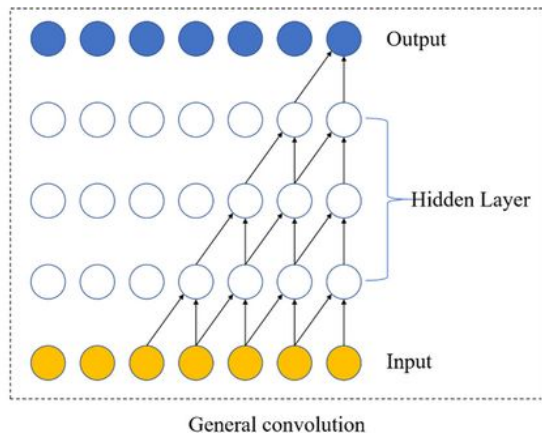
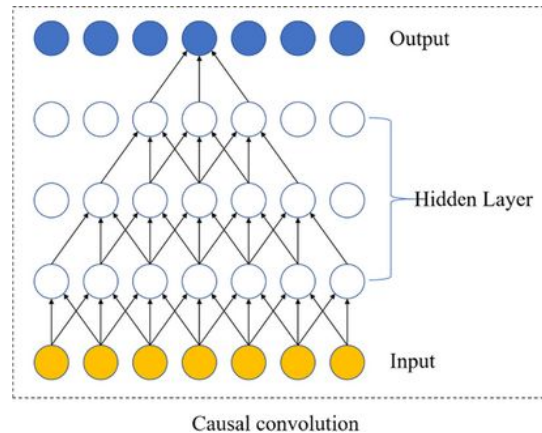
Convolución Causal

- Una convolución es causal cuando y_t depende únicamente de:

$$x_t, x_{t-1}, \dots, x_{t-k+1}$$

nunca de valores futuros.

- Se implementa con **padding asimétrico**: se agregan $(k-1)$ ceros solo al inicio de la secuencia, nunca al final
- Así se garantiza que el modelo respete el orden temporal y sea válido para forecasting autoregresivo.
- Condición para usar una CNN en forecasting.



El límite de la Convolución Causal simple

- Con solo causalidad, el receptive field (RF) de L capas con kernel k crece así:

$$RF = 1 + L \cdot (k - 1)$$

- Es crecimiento lineal.
- Una red tan profunda es computacionalmente costosa y sufre problemas de optimización (gradientes).
- Necesitamos una forma de crecer el RF mucho más rápido sin agregar tantas capas: **convolución dilatada**.

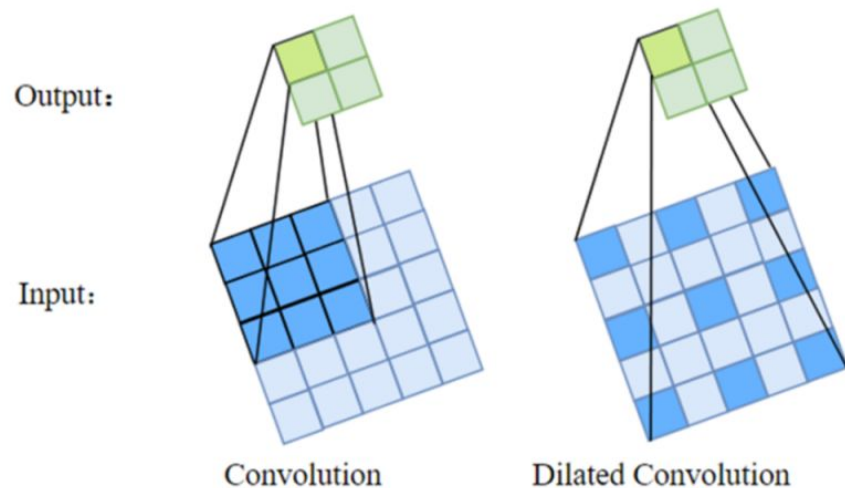
Convolución Dilatada

- Se introduce un parámetro de dilatación d : el filtro "salta" $d - 1$ posiciones entre cada peso.

- Fórmula general:

$$y_t = \sum_{i=0}^{k-1} w_i \cdot x_{t-d \cdot i}$$

- Con $d = 1$ es la convolución causal estándar, con $d = 2, 4, 8 \dots$ el filtro cubre un rango mucho mayor sin agregar parámetros.
- Al apilar capas con dilatación exponencialmente creciente el RF crece exponencialmente.



Esto es exactamente lo que permite que unas pocas capas (7-10) cubran cientos de pasos de historia

Convoluciones 1D

[LINK](#)



WaveNet

¿Qué es WaveNet?

- Arquitectura para generación de audio crudo (waveforms) punto por punto.
- Es el antecedente directo de la TCN: introduce por primera vez, en conjunto, la combinación de convoluciones causales + dilatadas apiladas para modelar secuencias largas.
- Diseñada originalmente como modelo generativo autoregresivo: predice la muestra x_t condicionada en todas las muestras anteriores:

$$p(x_t | x_1, \dots, x_{t-1})$$

WAVENET: A GENERATIVE MODEL FOR RAW AUDIO

Aäron van den Oord

Sander Dieleman

Heiga Zen[†]

Karen Simonyan

Oriol Vinyals

Alex Graves

Nal Kalchbrenner

Andrew Senior

Koray Kavukcuoglu

{avdnoord, sedielem, heigazen, simonyan, vinyals, graves, nalk, andrewsenior, korayk}@google.com
Google DeepMind, London, UK

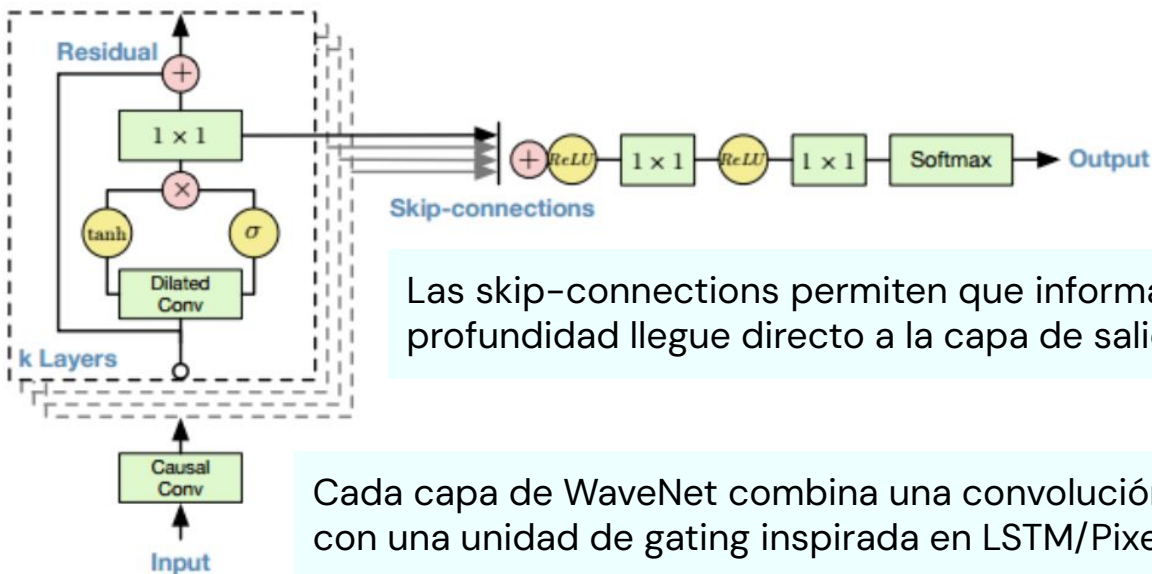
[†] Google, London, UK

ABSTRACT

This paper introduces WaveNet, a deep neural network for generating raw audio waveforms. The model is fully probabilistic and autoregressive, with the predictive distribution for each audio sample conditioned on all previous ones; nonetheless we show that it can be efficiently trained on data with tens of thousands of samples per second of audio. When applied to text-to-speech, it yields state-of-the-art performance, with human listeners rating it as significantly more natural sounding than the best parametric and concatenative systems for both English and Mandarin. A single WaveNet can capture the characteristics of many different speakers with equal fidelity, and can switch between them by conditioning on the speaker identity. When trained to model music, we find that it generates novel and often highly realistic musical fragments. We also show that it can be employed as a discriminative model, returning promising results for phoneme recognition.

Arquitectura de WaveNet

Cada bloque tiene dos salidas: una conexión residual y una conexión de skip (todas las skip-connections de todos los bloques se suman al final para producir la salida)



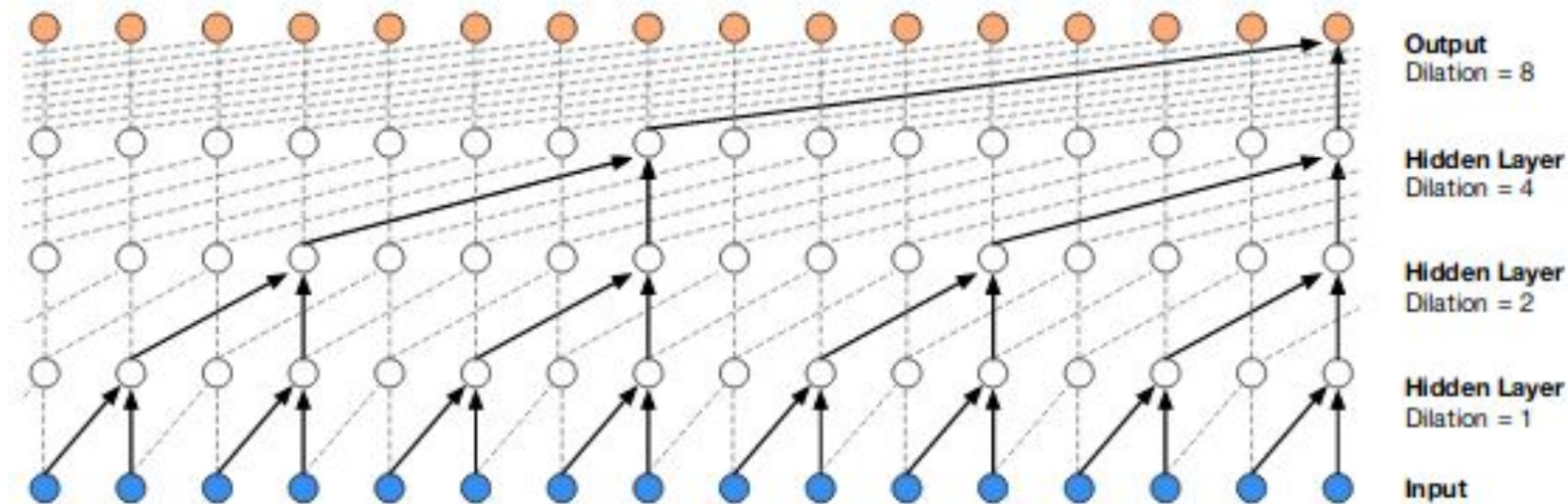
Las skip-connections permiten que información de cualquier profundidad llegue directo a la capa de salida.

Cada capa de WaveNet combina una convolución dilatada con una unidad de gating inspirada en LSTM/PixelCNN.

Dilatación en WaveNet

- WaveNet no solo aumenta la dilatación monótonamente; la organiza en ciclos repetidos: 1, 2, 4, ..., 512, 1, 2, 4, ..., 512, ...
- Cada ciclo cubre el mismo rango temporal completo, pero apilar varios ciclos aumenta la capacidad de representación sin perder cobertura del receptive field.
- Con 10 capas por ciclo (dilatación hasta 512) y varios ciclos apilados, el RF alcanza miles de pasos con relativamente pocos parámetros.

WaveNet como Generador Autoregresivo



En generación de audio, WaveNet predice una muestra a la vez y realimenta su propia salida como entrada para el siguiente paso (igual que un LSTM en modo generación).

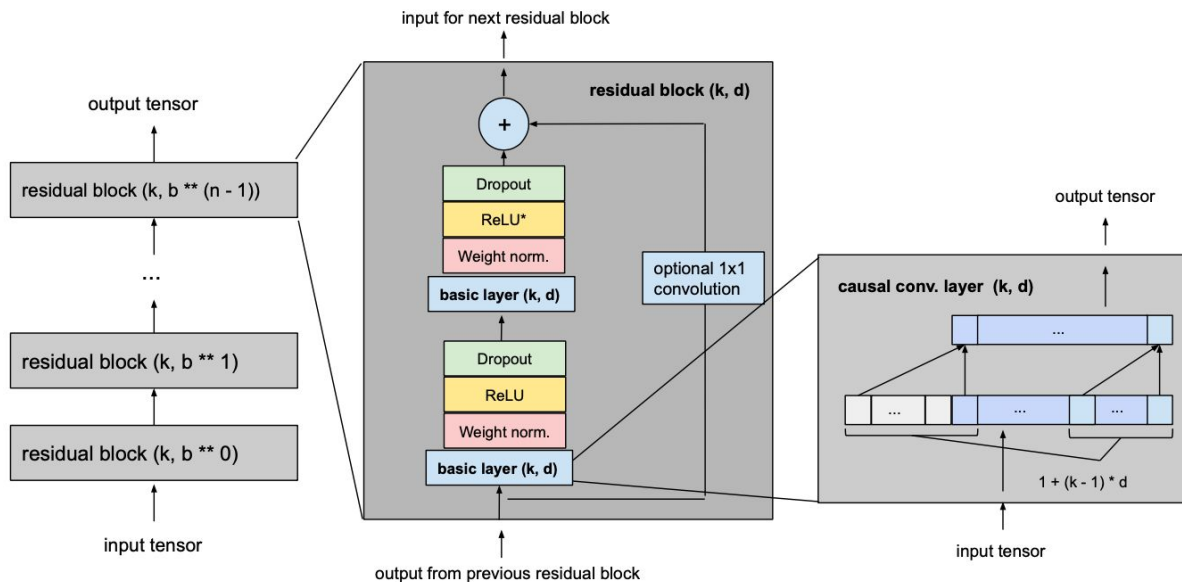


Esto hace la inferencia secuencial y lenta (miles de pasos por segundo de audio), a pesar de que el entrenamiento sí es paralelizable.

Esta arquitectura, simplificada y generalizada tiene el nombre **TCN**, una arquitectura genérica para cualquier tarea de secuencias (no solo audio)

Temporal Convolutional Networks (TCN)

¿Qué es una TCN?

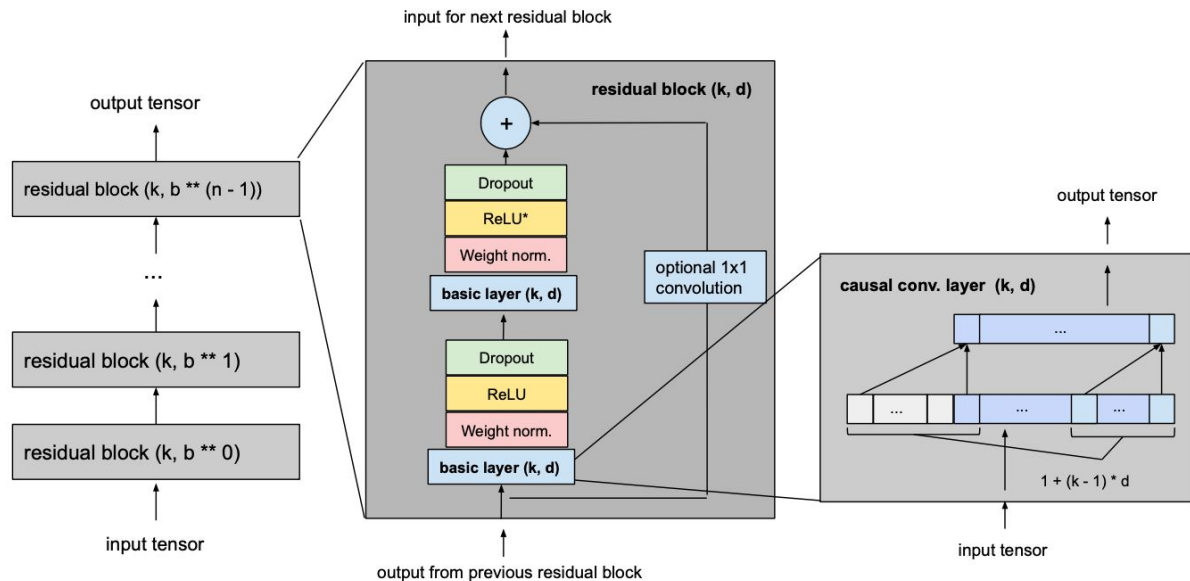


Alternativa genérica a RNN/LSTM para modelado de secuencias

Componentes: convoluciones causales + dilatadas + conexiones residuales + normalización de pesos + dropout

Produce una secuencia de salida de la misma longitud que la entrada (arquitectura "seq2seq")

Arquitectura



3) Para forecasting multi-horizonte, la capa final suele ser una capa densa que mapea el último estado temporal a un vector de H predicciones (enfoque "direct multi-step"), evitando así la propagación de errores del enfoque recursivo.

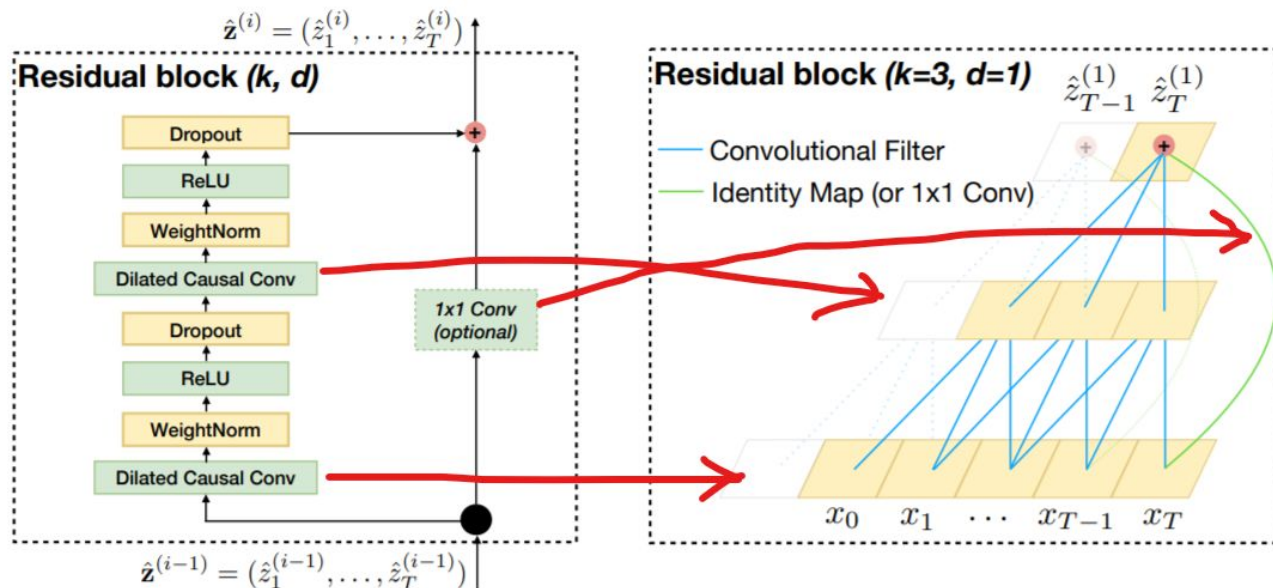
1) Es una pila de bloques residuales con dilatación creciente (1, 2, 4, 8, ...)

2) La salida final se toma en la última posición temporal (para forecasting de un solo paso) o se proyecta a un horizonte H (para forecasting multi-paso).

Entrenamiento
paralelizable.

Gradientes más
estables.

Bloque Residual Temporal



En lugar de forzar a que cada capa aprenda la transformación completa, se le pide que aprenda el "residuo" (la diferencia) respecto a la entrada.

Esto facilita la propagación del gradiente en redes profundas, un problema que las RNN/LSTM sufren con secuencias largas

Cada bloque contiene: 2 capas de convolución dilatada causal, activación (ReLU), dropout, repetido dos veces

Conexión residual: $\text{salida} = \text{Activación}(F(x) + x)$, con una convolución 1x1 si las dimensiones no coinciden

Permite apilar muchas capas sin degradación del gradiente

TCN como Global Forecasting Model

- La TCN se entrena sobre múltiples series simultáneamente.
- Las covariables estáticas (ej. categoría de producto) se incorporan concatenando embeddings a la entrada de cada paso temporal.
- Las covariables dinámicas conocidas a futuro (ej. calendario de feriados) se concatenan también como canales adicionales.
- Sus pesos son compartidos por diseño (el mismo filtro se aplica a cualquier serie que se le presente). Esto la diferencia de un modelo estadístico clásico ajustado serie por serie.

TCN

[LINK](#)



Ejercicio, TCN

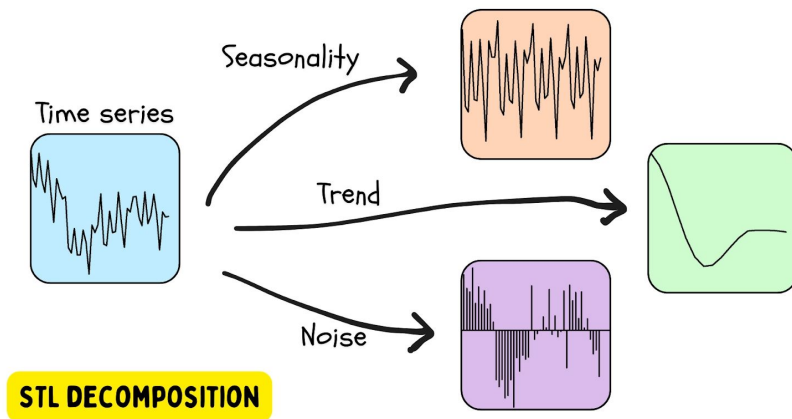
[LINK](#)



Descomposición temporal

Idea

- Enfoque clásico (ej. STL):
serie = Tendencia + Estacionalidad + Residuo
- **Arquitecturas modernas:** en lugar de que una sola red aprenda todo mezclado, se diseñan componentes especializados.
- **Ventaja: interpretabilidad**, se puede inspeccionar qué aprendió cada componente



La idea es llevar esto dentro de la arquitectura de red neuronal, en lugar de aplicarla como preprocesamiento externo.

STL

[LINK](#)



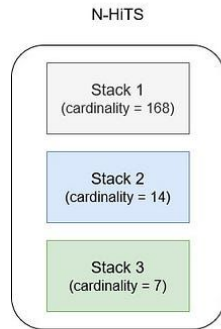
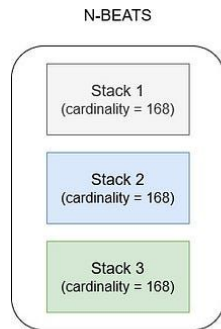
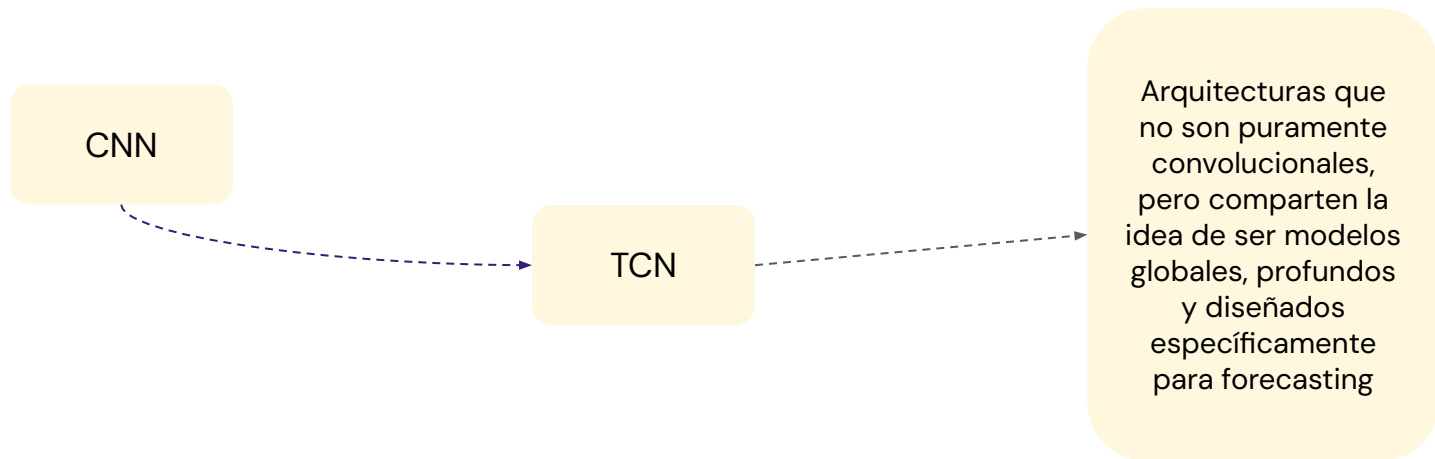
Descomposición como sesgo inductivo en redes neuronales

- Sesgo inductivo: restricciones de diseño que guían al modelo hacia soluciones más plausibles.
- Forzar a una parte de la red a modelar solo tendencia (funciones suaves/polinomiales) y a otra solo estacionalidad (funciones periódicas) mejora la generalización.
- Esta idea es el fundamento directo de N-BEATS.
- Una red totalmente libre **aprenda todo desde cero requiere muchos datos y es propensa a sobreajuste**. Al imponer una estructura se reduce el espacio de hipótesis a soluciones más razonables para forecasting, mejorando la generalización con menos datos.

Arquitecturas basadas en descomposición

¿Cómo se implementa esta idea de descomposición dentro de una red profunda moderna?

N-BEATS y N-HiTS, dos arquitecturas que llevan este principio al extremo mediante "basis expansion"

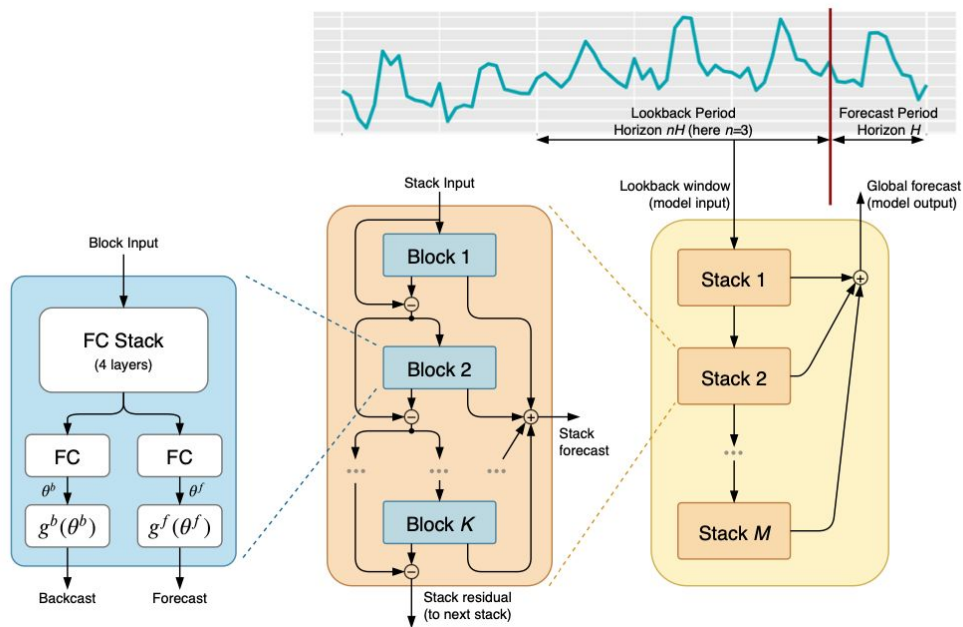


Arquitecturas modernas: N-BEATS y N-HiTS

N-BEATS: visión general

- Arquitectura puramente basada en capas totalmente conectadas (no convolucional, no recurrente).
- Estructura en "bloques" y "stacks" apilados, cada uno refinando el residuo del anterior.

N-BEATS al no usar RNN o CNN: demuestra que, con el diseño arquitectónico correcto, capas densas simples pueden alcanzar (e incluso superar) el desempeño de arquitecturas más complejas..

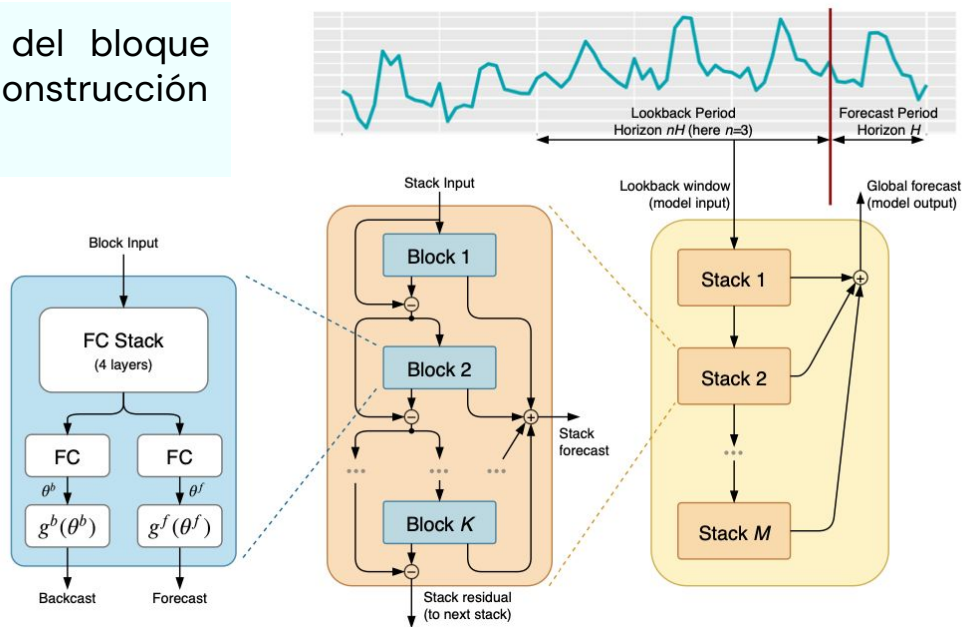


N-BEATS

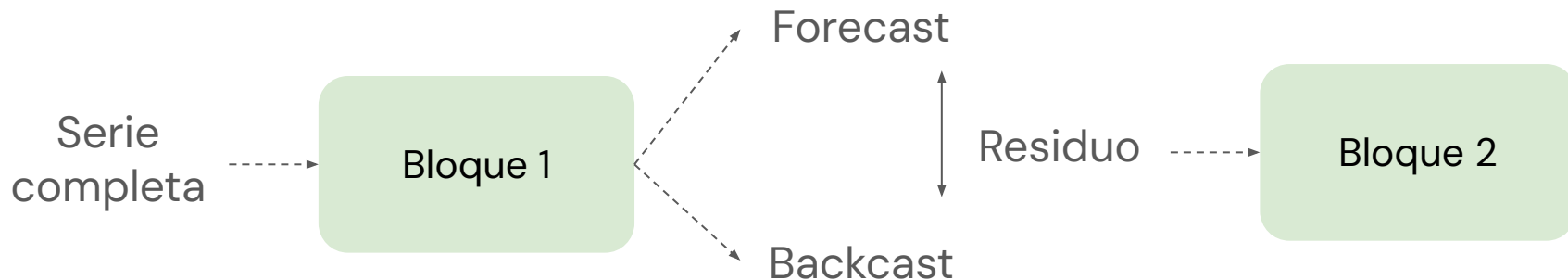
Cada bloque recibe la serie (o el residuo del bloque anterior) y produce dos salidas: backcast (reconstrucción del pasado) y forecast (predicción del futuro)

El backcast se resta de la entrada antes de pasar al siguiente bloque "doble residuo" (a través de bloques y a través de stacks)

Los forecasts de todos los bloques se suman para producir la predicción final



N-BEATS



Similar a gradient boosting: **cada bloque corrige el error del anterior**, pero aquí sucede dentro de una única red entrenada de punta a punta.

Ejemplo análogo a boosting: el bloque 1 podría capturar la tendencia general, dejando como residuo las fluctuaciones estacionales, que el bloque 2 se especializa en modelar, y así sucesivamente.

N-HiTS

- Neural Hierarchical Interpolation for Time Series (Challu et al., 2022)
- N-BEATS tiene dificultades con horizontes de forecast muy largos (alto costo computacional y de memoria)
- N-HiTS: cada stack opera a una tasa de muestreo distinta (submuestreo/pooling en distintas escalas temporales)

N-HiTS: Neural Hierarchical Interpolation for Time Series Forecasting

Cristian Challu,^{1*} Kin G. Olivares,^{1*} Boris N. Oreshkin,² Federico Garza,³ Max Mergenthaler-Canseco,³ Artur Dubrawski,¹

¹Auton Lab, School of Computer Science, Carnegie Mellon University, Pittsburgh, PA, USA

²Unity Technologies, Labs, Montreal, QC, Canada

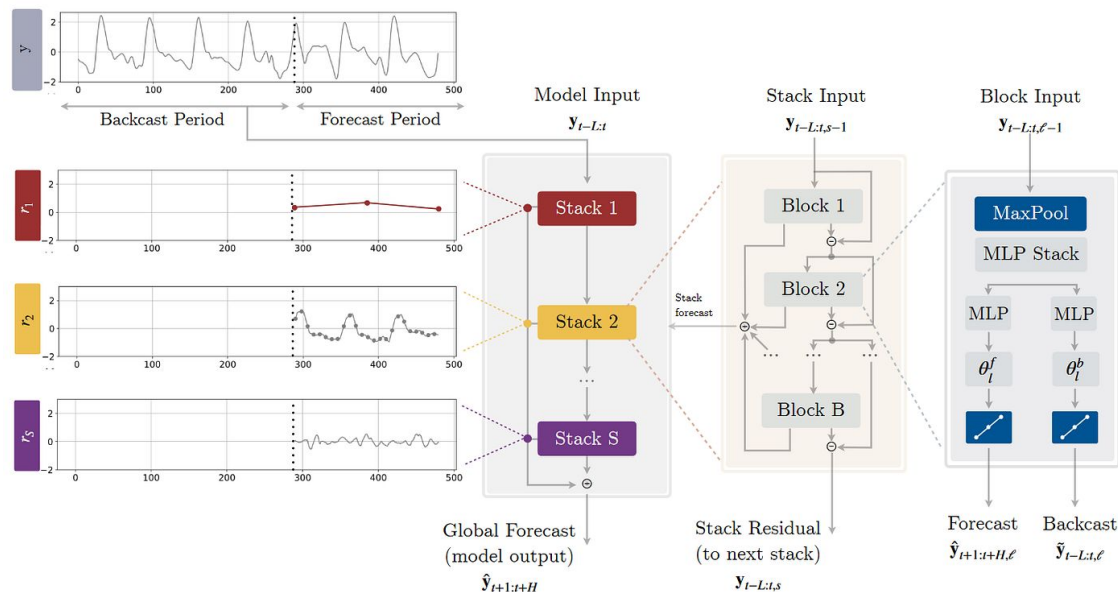
³Nixtla, Pittsburgh, PA, USA

{cchallu, kdgutier, awd}@cs.cmu.edu, boris.oreshkin@unity3d.com, {federico, max}@nixtla.io

Abstract

Recent progress in neural forecasting accelerated improvements in the performance of large-scale forecasting systems. Yet, long-horizon forecasting remains a very difficult task. Two common challenges afflicting the task are the volatility of the predictions and their computational complexity. We introduce N-HiTS, a model which addresses both challenges by incorporating novel hierarchical interpolation and multi-rate data sampling techniques. These techniques enable the proposed method to assemble its predictions sequentially, emphasizing components with different frequencies and scales while decomposing the input signal and synthesizing the forecast. We prove that the hierarchical interpolation technique can efficiently approximate arbitrarily long horizons in the presence of smoothness. Additionally, we conduct extensive large-scale dataset experiments from the long-horizon forecasting literature, demonstrating the advantages of our method over the state-of-the-art methods, where N-HiTS provides an average accuracy improvement of almost 20% over the latest Transformer architectures while reducing the computation time by an order of magnitude (50 times). Our code is available at <https://github.com/Nixtla/neuralforecast>.

N-HiTS: eficiencia en horizontes largos



Al usar pooling multi-escala, N-HiTS reduce el número de parámetros efectivos necesarios para horizontes largos

Reporta mejoras significativas de velocidad de entrenamiento e inferencia frente a N-BEATS y modelos basados en Transformers, manteniendo precisión competitiva

Especialmente relevante en escenarios industriales con horizontes de semanas o meses

N-BEATS vs N-HITS

[LINK](#)



Ejercicio, N-BEATS vs N-HITS

[LINK](#)

